МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

ИИТУС
БГТУ им. В.Г. Шухова

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Лабораторная работа №2

по дисциплине: Тестирование программных систем
тема: «Интеграционное и API-тестирование»

Выполнила: ст. группы ПВ-233
Скиба Мария Сергеевна

Проверил:
Доденко Олег Павлович

Белгород, 2026 г.

Цель: Освоить тестирование REST API на примере открытого публичного сервиса: научиться строить в Postman коллекцию запросов, писать автоматические проверки и анализировать расхождения между документацией и фактическим поведением API.

Задания

1. Создайте в Postman коллекцию LabAPI_ с переменными окружения (baseUrl, token, bookingId) и папками по ресурсам: Auth, Booking, Health.

2. Добавьте запросы ко всем эндпоинтам из таблицы. Для каждого запроса напишите описание: цель, входные данные, ожидаемый результат.

3. Автоматизируйте получение токена. В запросе POST /auth скриптом во вкладке Tests сохраните токен в переменную окружения. В запросе создания бронирования сохраните bookingid.

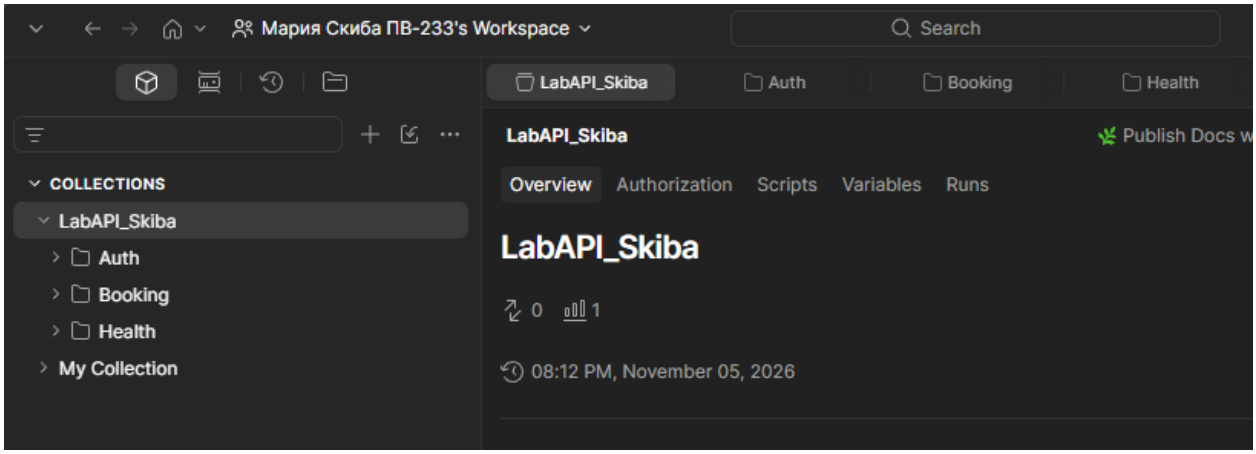
4. Составьте и выполните тест-кейсы в каждой категории: позитивные, негативные, граничные значения, коды ответов.

5. Для каждого запроса добавьте во вкладку Tests минимум 3 автоматические проверки.

6. Запустите всю коллекцию через Collection Runner и убедитесь, что цепочка сценариев работает без ручного вмешательства (токен и id передаются через переменные).

7. Фактическое и ожидаемое поведение. Это публичное учебное API, и его реальное поведение может отличаться от документации и от «правильного» REST (нестандартные коды для ошибок, код ответа для DELETE и т. д.). Оформите таблицу расхождений.

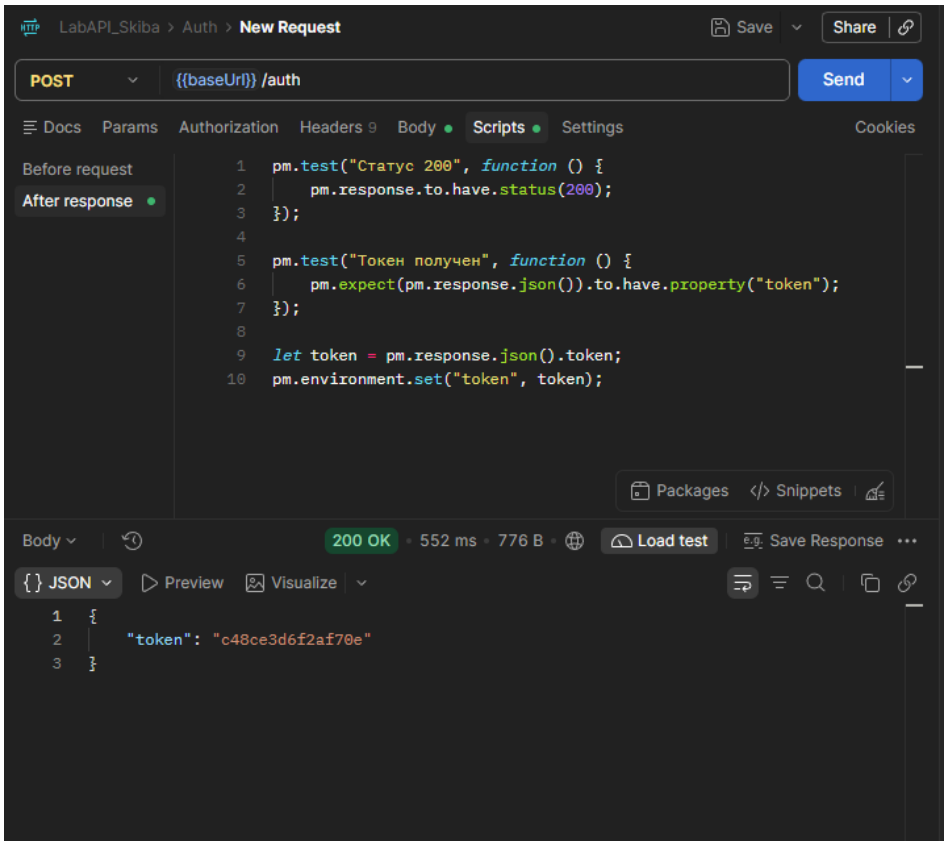
Задание 1.

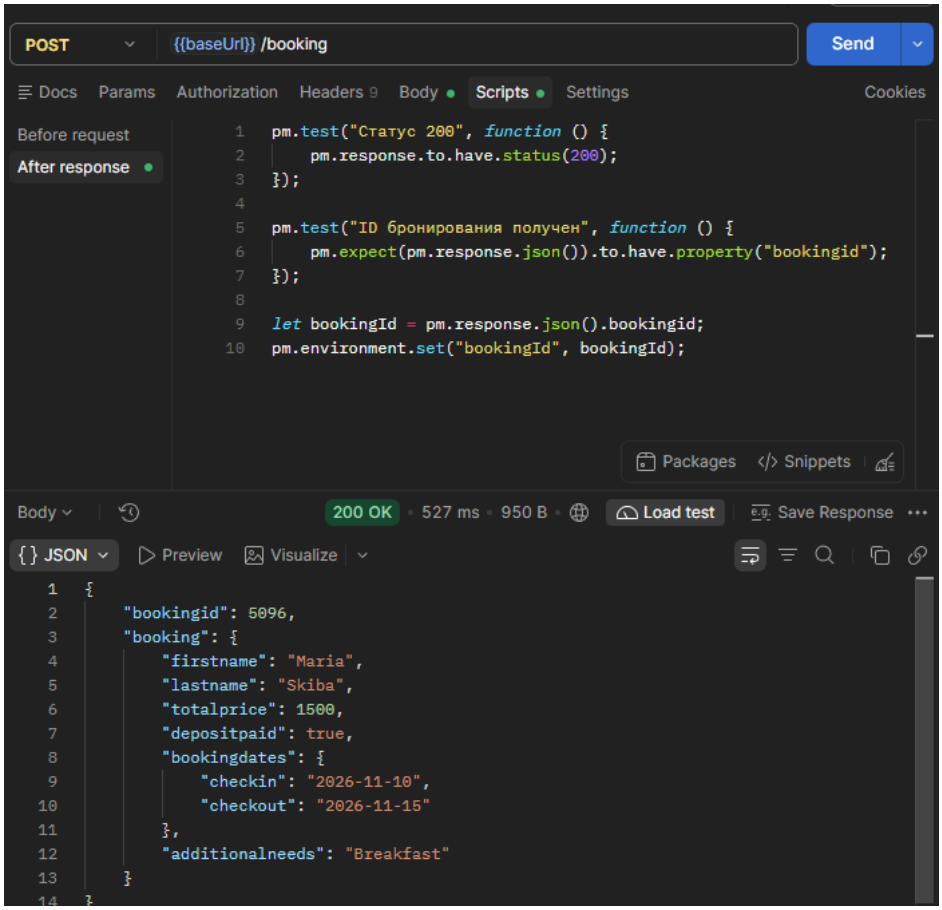


Задание 2.

Папка	Запрос (метод + URL)	Цель	Входные данные	Ожидаемый результат
Auth	POST {{baseUrl}}/auth	Получение токена авторизации.	JSON: {"username":"admin","password":"password123"}	Код 200 OK. Ответ содержит token, который сохраняется в переменную окружения.
Booking	GET {{baseUrl}}/booking	Получение списка всех бронирований.	Параметры не требуются.	Код 200 OK. Возвращается массив объектов с bookingid.
Booking	POST {{baseUrl}}/booking	Создание нового бронирования.	JSON: firstname, lastname, totalprice, depositpaid, bookingdates, additionalneeds.	Код 200 OK. Возвращается bookingid, который сохраняется в переменную окружения.
Booking	GET {{baseUrl}}/booking/{{bookingId}}	Получение информации о созданном бронировании.	Переменная пути bookingId.	Код 200 OK. Возвращаются данные бронирования.
Booking	PUT {{baseUrl}}/booking/{{bookingId}}	Полное обновление данных бронирования.	Header: Cookie: token={{token}}. JSON со всеми полями, изменено firstname на James.	Код 200 OK. Возвращаются обновлённые данные.
Booking	PATCH {{baseUrl}}/booking/{{bookingId}}	Частичное обновление бронирования.	Header: Cookie: token={{token}}. JSON: {"firstname":"Michael"}.	Код 200 OK. Имя изменено, остальные поля сохранены.
Booking	DELETE {{baseUrl}}/booking/{{bookingId}}	Удаление созданного бронирования.	Header: Cookie: token={{token}}.	Код 201 Created — особенность API.
Booking	GET {{baseUrl}}/booking/{{bookingId}}	Проверка удаления бронирования.	ID удалённого бронирования.	Код 404 Not Found.
Booking	DELETE {{baseUrl}}/booking/1	Проверка запрета удаления без авторизации.	Заголовок авторизации отсутствует.	Код 403 Forbidden.
Health	GET {{baseUrl}}/ping	Проверка работоспособности сервиса.	Параметры отсутствуют.	Код 201 Created — особенность API. Время ответа менее 2000 мс.

Задание 3.


LabAPI_Skiba > Auth > New Request
Save
Share
POST
{{baseUrl}} /auth
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Before request
After response
pm.test("Статус 200", function () {
pm.response.to.have.status(200);
});
pm.test("Токен получен", function () {
pm.expect(pm.response.json()).to.have.property("token");
});
let token = pm.response.json().token;
pm.environment.set("token", token);
Packages
Snippets
Body
200 OK
552 ms
776 B
Load test
Save Response
{} JSON
Preview
Visualize
{
"token": "c48ce3d6f2af70e"
}


POST
{{baseUrl}} /booking
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Before request
After response
pm.test("Статус 200", function () {
pm.response.to.have.status(200);
});
pm.test("ID бронирования получен", function () {
pm.expect(pm.response.json()).to.have.property("bookingid");
});
let bookingId = pm.response.json().bookingid;
pm.environment.set("bookingId", bookingId);
Packages
Snippets
Body
200 OK
527 ms
950 B
Load test
Save Response
{} JSON
Preview
Visualize
{
"bookingid": 5096,
"booking": {
"firstname": "Maria",
"lastname": "Skiba",
"totalprice": 1500,
"depositpaid": true,
"bookingdates": {
"checkin": "2026-11-10",
"checkout": "2026-11-15"
},
"additionalneeds": "Breakfast"
}
}

Задание 4.

ID	Эндпоинт	Тип	Входные данные	Ожидаемый код	Фактический код	Статус
TC-01	POST /booking	Позитивный	Валидный JSON (создание)	200	200	Passed
TC-02	GET /booking/	Позитивный	Существующий bookingId	200	200	Passed
TC-03	PUT /booking/	Позитивный	Токен + валидный JSON	200	200	Passed
TC-04	PATCH /booking/	Позитивный	Токен + {"firstname":"James"}	200	200	Passed
TC-05	DELETE /booking/	Позитивный	Токен + существующий ID	201	201	Passed
TC-06	GET /booking/	Негативный	ID удалённого бронирования	404	404	Passed
TC-07	POST /auth	Негативный	Неверный пароль wrongpass	200	200	Passed
TC-08	DELETE /booking/1	Негативный	Без токена авторизации	403	403	Passed
TC-09	PUT /booking/1	Негативный	Неверный токен token=123	403	403	Passed
TC-10	GET /booking/999999	Негативный	Несуществующий ID	404	404	Passed
TC-11	POST /booking	Негативный	Некорректный JSON	400	500	Failed
TC-12	POST /booking	Негативный	totalprice: "сто"	400	500	Failed

TC-13	POST /booking	Негативный	Отсутствует поле firstname	400	500	Failed
TC-14	POST /booking	Граничный	firstname: ""	400	200	Failed
TC-15	POST /booking	Граничный	firstname длиной 1000 символов	200	200	Passed
TC-16	POST /booking	Граничный	totalprice: 0	200	200	Passed
TC-17	POST /booking	Граничный	totalprice: -100	400	200	Failed
TC-18	POST /booking	Граничный	totalprice: 999999999	200	200	Passed
TC-19	POST /booking	Граничный	checkout раньше checkin	400	200	Failed
TC-20	POST /booking	Граничный	checkin: "2024-13-45"	400	500	Failed

Задание 5.

1. POST Auth (Авторизация)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Есть токен", () =>
pm.expect(pm.response.json()).to.have.property("token"));

pm.environment.set("token", pm.response.json().token);
```

2. POST Booking (Создание бронирования)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Есть bookingid", () =>
pm.expect(pm.response.json()).to.have.property("bookingid"));
pm.test("Имя сохранилось верно", () =>
    pm.expect(pm.response.json().booking.firstname).to.eql("Maria"));

pm.environment.set("bookingId", pm.response.json().bookingid);
```

3. GET Booking (Получение бронирования)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Имя совпадает", () =>
    pm.expect(pm.response.json().firstname).to.eql("Maria"));
pm.test("Фамилия совпадает", () =>
    pm.expect(pm.response.json().lastname).to.eql("Skiba"));
```

4. PUT Booking (Полное обновление)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Имя обновилось на James", () =>
    pm.expect(pm.response.json().firstname).to.eql("James"));
pm.test("Фамилия не изменилась", () =>
    pm.expect(pm.response.json().lastname).to.eql("Skiba"));
```

5. PATCH Booking (Частичное обновление)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Имя изменилось на Michael", () =>
    pm.expect(pm.response.json().firstname).to.eql("Michael"));
pm.test("Фамилия не изменилась", () =>
    pm.expect(pm.response.json().lastname).to.eql("Skiba"));
```

6. DELETE Booking (Удаление)

```
pm.test("Статус 201 Created", () => pm.response.to.have.status(201));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Ответ содержит Created", () =>
    pm.expect(pm.response.text()).to.include("Created"));
```

7. GET Ping (Проверка работоспособности)

```
pm.test("Статус 201 Created", () => pm.response.to.have.status(201));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Ответ содержит Created", () =>
    pm.expect(pm.response.text()).to.include("Created"));
```

8. DELETE No Auth (Удаление без авторизации)

```
pm.test("Статус 403 Forbidden", () => pm.response.to.have.status(403));
```

```
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Ответ содержит Forbidden", () =>
    pm.expect(pm.response.text()).to.include("Forbidden"));
```

9. GET Booking List (Список бронирований)

```
pm.test("Статус 200", () => pm.response.to.have.status(200));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Ответ является массивом", () =>
    pm.expect(pm.response.json()).to.be.an("array"));
```

10. GET Deleted Booking (Проверка удаления)

```
pm.test("Статус 404 Not Found", () => pm.response.to.have.status(404));
pm.test("Время ответа < 2000 мс", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("Бронирование не найдено", () =>
    pm.expect(pm.response.text()).to.include("Not Found"));
```

Задание 7.

№	Запрос	Ожидалось	Получено	Вывод
1	DELETE /booking/	200 OK или 204 No Content	201 Created	Особенность API. При успешном удалении возвращается код 201 вместо 200 или 204.
2	POST /booking (невалидный JSON)	400 Bad Request	500 Internal Server Error	Дефект. Некорректный JSON вызывает внутреннюю ошибку сервера.

3	POST /booking (нет firstname)	400 Bad Request	500 Internal Server Error	Дефект. Отсутствует корректная обработка обязательных полей.
4	POST /booking (totalprice — строка)	400 Bad Request	500 Internal Server Error	Дефект. Неправильный тип данных приводит к ошибке сервера.
5	POST /booking (невалидная дата)	400 Bad Request	500 Internal Server Error	Дефект. Некорректная дата вызывает ошибку 500.
6	POST /booking (totalprice = -100)	400 Bad Request	200 OK	Дефект. API принимает отрицательную стоимость бронирования.
7	POST /booking (firstname = "")	400 Bad Request	200 OK	Особенность. API допускает пустое имя клиента.
8	POST /booking (строка 1000+ символов)	400 Bad Request при ограничении длины	200 OK	Особенность. API принимает очень длинные строки.

9	POST /booking (checkout раньше checkin)	400 Bad Request	200 OK	Дефект. Не проверяется правильность последовательности дат.
10	POST /auth (неверный пароль)	401 Unauthorized	200 OK, Bad credentials	Особенность. Ошибка авторизации возвращается с кодом 200.
11	PUT/PATCH/DELETE (без токена)	401 Unauthorized	403 Forbidden	Особенность. Отсутствие авторизации обрабатывается кодом 403.
12	PUT/PATCH/DELETE (неверный токен)	401 Unauthorized	403 Forbidden	Особенность. Неверный токен вызывает ответ 403 вместо 401.

Баг-репорт 1 (Серьёзность: Major)

- Шаги: отправить POST /booking с некорректным типом данных {"totalprice": "сто"}, остальные обязательные поля заполнить корректно.
- Ожидаемый результат: код 400 Bad Request с указанием ошибки валидации.
- Фактический результат: код 500 Internal Server Error.

Баг-репорт 2 (Серьёзность: Major)

- Шаги: отправить POST /booking без обязательного поля firstname, остальные поля заполнить корректно.

- Ожидаемый результат: код 400 Bad Request.
- Фактический результат: код 500 Internal Server Error.

Баг-репорт 3 (Серьёзность: Minor)

- Шаги: отправить POST /booking с totalprice = -100.
- Ожидаемый результат: код 400 Bad Request, поскольку стоимость бронирования не может быть отрицательной.
- Фактический результат: код 200 OK, бронирование успешно создаётся.

Вывод: Освоила тестирование REST API на примере открытого публичного сервиса: научилась строить в Postman коллекцию запросов, писать автоматические проверки и анализировать расхождения между документацией и фактическим поведением API.